Instituto Politécnico Nacional
Centro de Investigación en Computación

Clasificación Inteligente de Patrones (A25)
Prof.: Dr. Antonio Alarcón Paredes

Tarea 5

Luis Angel Rojo Chavez
arojocz@gmail.com
lrojoc2024@cic.ipn.mx

26 de marzo de 2025

Métricas de evaluación multiclase

Calcular las métricas de evaluación, basado en una matriz de confusión MULTICLASE, como la vista en la clase de hoy. Los elementos a considerar son los siguientes:

1. Dataset Wine de UCI: https://archive.ics.uci.edu/dataset/109/wine
2. Método de validación: Leave-one-out
3. Clasificadores de Scikit-learn: KNeighbors (k=3) y GaussianNB
4. Métricas: Sensibilidad, especificidad, balanced accuracy, precisión y F1-Score
5. Métricas finales, con average: Macro, Weighted y Micro.

Como el dataset tiene 3 clases, se entregarán las 3 matrices de confusión que se generan con ambos clasificadores (6 en total), las métricas para cada matriz de confusión, y el desempeño final obteniendo el promedio de las tres matrices de confisión por clasificador a partir de los métodos Macro, Weighted y Micro.

1. Código en Python

```python
import numpy as np
from sklearn.model_selection import LeaveOneOut
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix
from tabulate import tabulate
import matplotlib.pyplot as plt
import seaborn as sns

def model_choice(model_type):
    """Función para seleccionar el modelo a utilizar."""
    if model_type == 'knn':
        return KNeighborsClassifier(n_neighbors=3)
    elif model_type == 'gaussianNB':
```

```python
        return GaussianNB()
    else:
        raise ValueError("Ponlo bien")

def knn_or_nb(file, model_type='knn'):
    df = pd.read_csv(file, comment='@', header=None, delimiter=',')
    df_class = df.iloc[:, 0].values
    df_features = df.iloc[:, 1:].values
    np.set_printoptions(suppress=True)  # para no usar notación científica

    loo = LeaveOneOut()
    unique_classes = np.unique(df_class)
    y_true = []
    y_classified = []

    model = model_choice(model_type)

    for train_index, test_index in loo.split(df_features):
        X_train, X_test = df_features[train_index], df_features[test_index]
        Y_train, Y_test = df_class[train_index], df_class[test_index]

        model.fit(X_train, Y_train)
        y_pred = model.predict(X_test)

        y_classified.append(int(y_pred))
        y_true.append(int(Y_test))

    conf_matrix = confusion_matrix(y_true, y_classified, labels=unique_classes)
    return conf_matrix, unique_classes

def plot_confusion_matrix(matrix, labels, title="Matriz de confusión"):
    plt.figure(figsize=(8, 6))
    sns.heatmap(matrix, annot=True, fmt='d', cmap='Blues', xticklabels=labels,
    yticklabels=labels)
    plt.xlabel('Clasificación')
    plt.ylabel('Realidad')
    plt.title(title)
    plt.show()

def matrixxx(model_type='knn'):
    matrix, unique_classes = knn_or_nb('wine.data', model_type)
    print(f"Matriz de Confusión Completa ({model_type}):")
    print(matrix)

    # Plotting the 3x3 confusion matrix
```

```python
    plot_confusion_matrix(matrix, unique_classes,
    title=f"Matriz de confusión completa ({model_type})")

    confusion_matrices = np.zeros((len(matrix), 2, 2), dtype=int)
    metrics = []
    for i, cls in enumerate(unique_classes):
        TP = matrix[i, i]
        FN = np.sum(matrix[i, :]) - TP
        FP = np.sum(matrix[:, i]) - TP
        TN = np.sum(matrix) - (TP + FN + FP)
        confusion_matrices[i] = np.array([
            [TP, FN],
            [FP, TN]
        ])
        recall = TP / (TP + FN)
        spec = TN / (TN + FP)
        balanced_acc = (recall + spec) / 2
        precision = TP / (TP + FP)
        f1_score = 2 / ((1 / recall) + (1 / precision))
        metrics.append([recall, spec, balanced_acc, precision, f1_score])

    #métricas en tabulate
    print("\nMétricas por clase:")
    int_data = [[float(round(x, 5)) for x in row] for row in metrics]
    count = 0
    for row in int_data:
        row.insert(0, f"Clase{count+1}")
        count += 1

    headers = ["Clases", "Sensibilidad", "Especificidad", "Balanced Acc",
    "Precisión", "F1 score"]
    print(tabulate(int_data, headers=headers, tablefmt="grid"))

    average_data = [['Macro'], ['Weighted'], ['Micro']]
    macro_avg = np.mean(metrics, axis=0)
    for element in macro_avg:
        average_data[0].append(element)

    class_counts = np.sum(matrix, axis=1)  # Cantidad de valoress de cada clase
    weighted_avg = np.average(metrics, axis=0, weights=class_counts)
    for element in weighted_avg:
        average_data[1].append(element)

    TP_total = np.sum(np.diagonal(matrix))  # Sumar todas las verdaderas positivas
    FN_total = np.sum(np.sum(matrix, axis=1)) - TP_total
```

```python
    FP_total = np.sum(np.sum(matrix, axis=0)) - TP_total
    TN_total = np.sum(matrix) - (TP_total + FN_total + FP_total)

    # micro 2x2
    recall_micro = TP_total / (TP_total + FN_total)
    precision_micro = TP_total / (TP_total + FP_total)
    f1_micro = 2 / ((1 / recall_micro) + (1 / precision_micro))
    micro_avg = [recall_micro, precision_micro, (recall_micro +
    precision_micro) / 2, precision_micro, f1_micro]
    average_data[2].extend(micro_avg)

    print("\nPromedios de las métricas (Macro, Weighted, Micro):")
    print(tabulate(average_data, tablefmt="grid"))

    # Plotting class confusion matrices
    for i, cls in enumerate(unique_classes):
        class_matrix = confusion_matrices[i]
        print(f"Matriz de confusión de clase {cls}:")
        print(class_matrix)
        plot_confusion_matrix(class_matrix, ["Positive", "Negative"],
        title=f"Matriz de confusión para Clase {cls}")

model_type = 'gaussianNB'  # Cambiar entre 'knn' y 'gaussianNB'
matrixxx(model_type)
```

2. Fórmulas

$$\text{Accuracy} = \frac{\text{TP}+\text{TN}}{\text{TP}+\text{FP}+\text{FN}+\text{TN}} \qquad \text{Balanced Accuracy} = \frac{1}{K}\sum_{i=1}^{K}\text{Sensibility}(C_i)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP}+\text{FN}} \qquad \text{Precisión} = \frac{\text{TP}}{\text{TP}+\text{FP}}$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN}+\text{FP}} \qquad F_1\text{-Score} = \frac{2}{\frac{1}{\text{Recall}}+\frac{1}{\text{Precision}}}$$

3. Clasificador KNN

Usando 3 vecinos y `from sklearn.neighbors import KNeighborsClassifier` obtenemos las siguientes matrices de confusión y métricas.

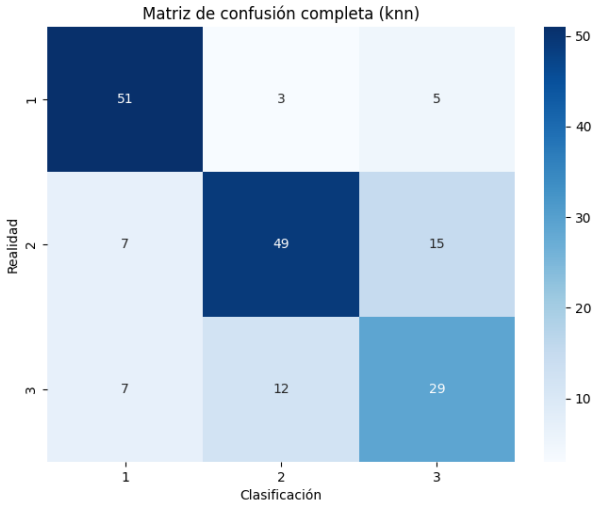

Figura 1: KNN matriz de confusión.

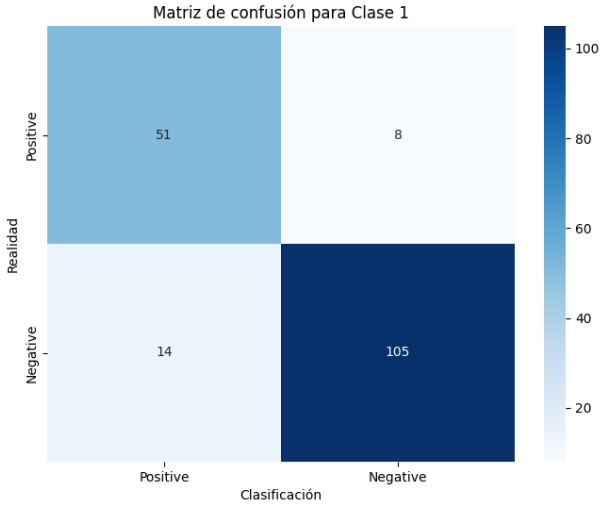

Figura 2: Matriz de confusión clase 1.

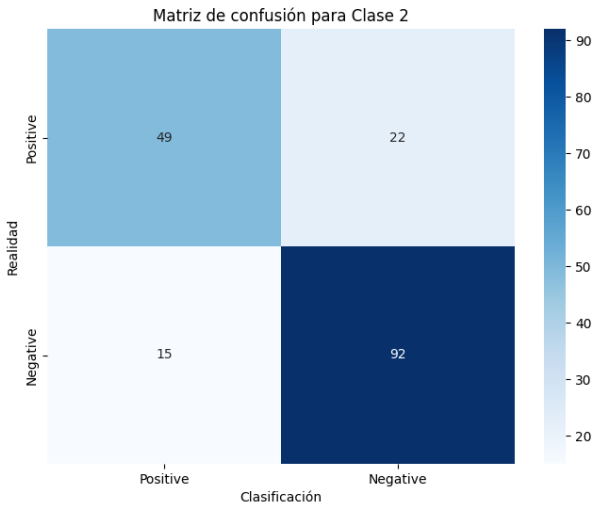

Figura 3: Matriz de confusión clase 2

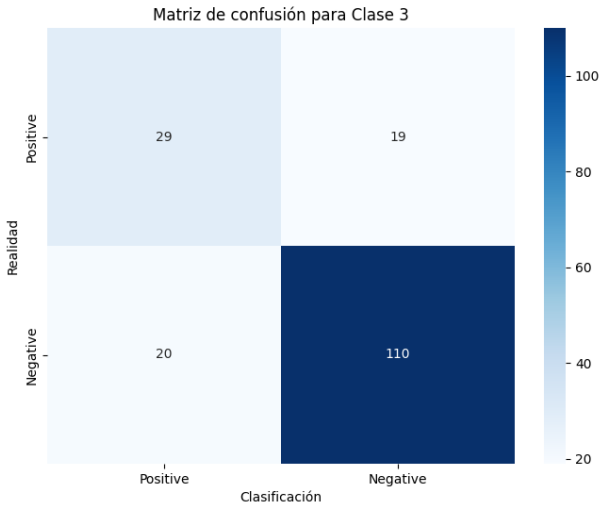

Figura 4: Matriz de confusión clase 3

```
Matriz de Confusión Completa (knn):
[[51  3  5]
 [ 7 49 15]
 [ 7 12 29]]
2025-03-26 09:49:44.613 Python[10258:500757] +[IMKClient subclass]: chose IMKClient_Modern
2025-03-26 09:49:44.613 Python[10258:500757] +[IMKInputSession subclass]: chose IMKInputSession_Modern

Métricas por clase:
+----------+----------------+-----------------+----------------+-------------+------------+
| Clases   |   Sensibilidad |   Especificidad |   Balanced Acc |   Precisión |   F1 score |
+==========+================+=================+================+=============+============+
| Clase1   |        0.86441 |         0.88235 |        0.87338 |     0.78462 |    0.82258 |
+----------+----------------+-----------------+----------------+-------------+------------+
| Clase2   |        0.69014 |         0.85981 |        0.77498 |     0.76562 |    0.72593 |
+----------+----------------+-----------------+----------------+-------------+------------+
| Clase3   |        0.60417 |         0.84615 |        0.72516 |     0.59184 |    0.59794 |
+----------+----------------+-----------------+----------------+-------------+------------+

Promedios de las métricas (Macro, Weighted, Micro):
+----------+----------+----------+----------+----------+----------+
| Macro    | 0.719571 | 0.862773 | 0.791172 | 0.714026 | 0.715482 |
+----------+----------+----------+----------+----------+----------+
| Weighted | 0.724719 | 0.863601 | 0.79416  | 0.725055 | 0.72345  |
+----------+----------+----------+----------+----------+----------+
| Micro    | 0.724719 | 0.724719 | 0.724719 | 0.724719 | 0.724719 |
+----------+----------+----------+----------+----------+----------+
Matriz de confusión de clase 1:
[[ 51   8]
 [ 14 105]]
Matriz de confusión de clase 2:
[[49 22]
 [15 92]]
Matriz de confusión de clase 3:
[[ 29  19]
 [ 20 110]]
arojocz@Luis-Rojo Tarea 5 %
```

Figura 5: Resultados para knn con 3 vecinos.

4. Gaussian Naive Bayes

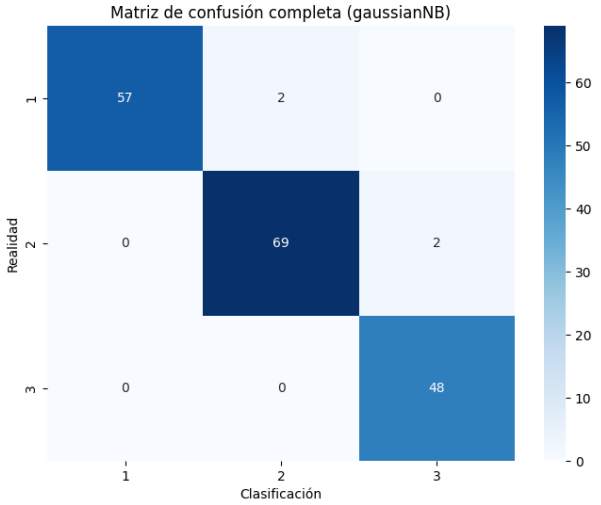

Figura 6: Matriz de confusión para GaussianNB

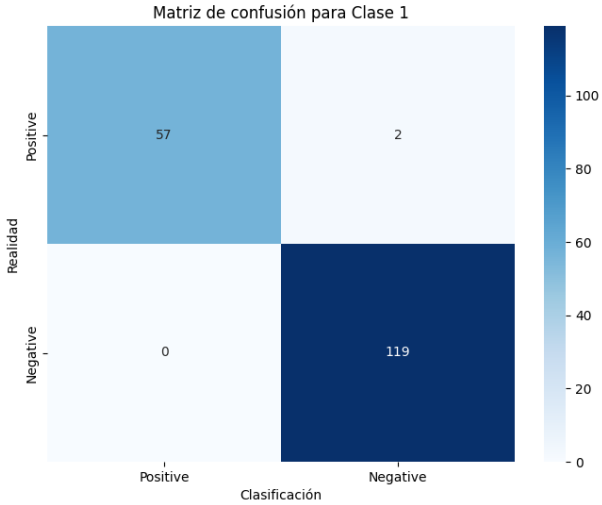

Figura 7: Matriz de confusión para la clase 1

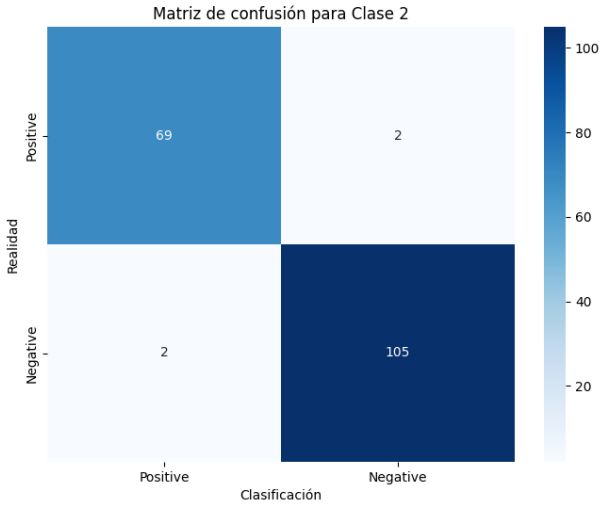

Figura 8: Matriz de confusión para la clase 2

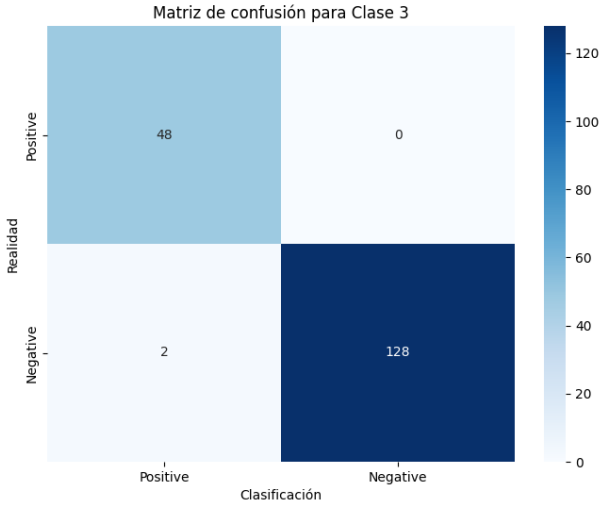

Figura 9: Matriz de clasificación para la clase 3

```
Matriz de Confusión Completa (gaussianNB):
[[57  2  0]
 [ 0 69  2]
 [ 0  0 48]]
2025-03-26 09:52:01.740 Python[10291:502445] +[IMKClient subclass]: chose IMKClient_Modern
2025-03-26 09:52:01.740 Python[10291:502445] +[IMKInputSession subclass]: chose IMKInputSession_Modern

Métricas por clase:
+----------+----------------+-----------------+----------------+-------------+------------+
| Clases   |   Sensibilidad |   Especificidad |   Balanced Acc |   Precisión |   F1 score |
+==========+================+=================+================+=============+============+
| Clase1   |        0.9661  |         1       |        0.98305 |     1       |    0.98276 |
+----------+----------------+-----------------+----------------+-------------+------------+
| Clase2   |        0.97183 |         0.98131 |        0.97657 |     0.97183 |    0.97183 |
+----------+----------------+-----------------+----------------+-------------+------------+
| Clase3   |        1       |         0.98462 |        0.99231 |     0.96    |    0.97959 |
+----------+----------------+-----------------+----------------+-------------+------------+

Promedios de las métricas (Macro, Weighted, Micro):
+----------+----------+----------+----------+----------+----------+
| Macro    | 0.979311 | 0.988641 | 0.983976 | 0.977277 | 0.97806  |
+----------+----------+----------+----------+----------+----------+
| Weighted | 0.977528 | 0.988396 | 0.982962 | 0.977978 | 0.977546 |
+----------+----------+----------+----------+----------+----------+
| Micro    | 0.977528 | 0.977528 | 0.977528 | 0.977528 | 0.977528 |
+----------+----------+----------+----------+----------+----------+
Matriz de confusión de clase 1:
[[ 57   2]
 [  0 119]]
Matriz de confusión de clase 2:
[[ 69   2]
 [  2 105]]
Matriz de confusión de clase 3:
[[ 48   0]
 [  2 128]]
arojocz@Luis-Rojo Tarea 5 %
```

Figura 10: Resultados para GaussianNB